🐧 Linux WSL2 | 🐳 Docker | ⚙️ Nginx | 📄 HTML | 🎓 Formation TSSR

Projet technique Linux & Docker Préparation TSSR

Mise en place d'un environnement Linux sous Windows via WSL2 et déploiement d'un serveur web conteneurisé avec Docker et Nginx — documentation technique étape par étape.

NOM DU PROJET	Projet Docker TSSR
TECHNOLOGIES	Linux · WSL2 · Docker · Nginx
OBJECTIF	Admin système & réseau
CADRE	Préparation formation TSSR
ÉTAPES DOCUMENTÉES	22 étapes

RÉALISÉ PAR	Stagiaire TSSR
DATE	2025
FORMATION	TSSR — Technicien Supérieur Systèmes et Réseaux

📋 Sommaire

Table des matières — 22 étapes documentées

DOCUMENT

01 Introduction — Contexte, objectifs et organisation

ENVIRONNEMENT LINUX — WSL2 & UBUNTU

A Vérification de WSL
B Lancement d'Ubuntu
C Informations de base Linux
D Mise à jour du système Linux
E Vérification de la version du noyau Linux
F1 Vérification de la distribution Linux

DOCKER — INSTALLATION & COMMANDES

F2 Démarrage de Docker Desktop
G Test de l'installation Docker — hello-world
H Vérification de la version Docker
I Informations détaillées Docker
J Vérification des images Docker
K1 Conteneurs Docker actifs — docker ps
K2 Tous les conteneurs Docker — docker ps -a
K3 Arrêt d'un conteneur Docker — docker stop

CONTENEURS & FICHIERS

L Lancement d'un conteneur Ubuntu interactif
M Mise à jour et installation dans le conteneur
N Création d'un fichier texte dans le conteneur
O Lecture du fichier dans le conteneur

SERVEUR WEB NGINX

P Déploiement d'un serveur web Nginx
Q Création du dossier du site personnalisé
R Création de la page HTML personnalisée
S Déploiement de la page personnalisée avec Nginx

ANNEXE

🖥️ Environnement technique personnel

BILAN DU PROJET

⚠ Difficultés rencontrées

★ Compétences acquises

✓ Conclusion

📖 Glossaire technique

📋 Sommaire

Vue d'ensemble de la documentation technique — 22 étapes documentées

PROJET TECHNIQUE TSSR

- 01 Introduction — Contexte, objectifs et organisation
- A Vérification de WSL
- B Lancement d'Ubuntu
- C Informations de base Linux
- D Mise à jour du système Linux
- E Vérification de la version du noyau Linux
- F1 Vérification de la distribution Linux
- F2 Démarrage de Docker Desktop
- G Test Docker — hello-world
- H Vérification de la version Docker
- I Informations détaillées Docker
- J Vérification des images Docker
- K1 Conteneurs actifs — docker ps
- K2 Tous les conteneurs — docker ps -a
- K3 Arrêt d'un conteneur — docker stop
- L Conteneur Ubuntu interactif
- M Mise à jour et installation dans le conteneur
- N Création d'un fichier texte dans le conteneur
- O Lecture du fichier dans le conteneur
- P Déploiement du serveur web Nginx
- Q Création du dossier du site personnalisé
- R Création de la page HTML personnalisée
- S Déploiement de la page avec Nginx

ANNEXE

- 🖥️ Environnement technique personnel

BILAN

- ⚠ Difficultés rencontrées
- ★ Compétences acquises
- 📖 Glossaire technique

Conclusion

🎯

PARTIE PRINCIPALE — Projet Technique TSSR

Documentation détaillée de toutes les étapes réalisées dans le cadre du projet de préparation à la formation TSSR.

01 Introduction

Contexte, objectifs et organisation du document

Ce document constitue la **documentation technique complète** d'un projet réalisé en autonomie dans le cadre de la préparation à la formation **TSSR (Technicien Supérieur Systèmes et Réseaux)**. Il retrace, étape par étape, toutes les manipulations effectuées pour découvrir, installer, configurer et utiliser un environnement Linux sous Windows (via WSL2), puis déployer un serveur web conteneurisé avec Docker et Nginx.

Chaque étape précise quel programme ouvrir, quel terminal utiliser, quelles commandes saisir, ce qu'elles font, le résultat attendu, et un espace réservé pour les captures d'écran.

Note importante : Ce document contient deux parties distinctes qui ne doivent pas être confondues. La **partie principale** présente le projet TSSR concret réalisé. Une **section séparée**, clairement identifiée plus bas, présente uniquement les outils présents sur l'environnement personnel de travail — sans signifier que tous ont été mobilisés dans ce projet.

SYSTÈME HÔTE

Windows 10/11 avec WSL2 activé

ENVIRONNEMENT LINUX

Ubuntu via WSL2

TECHNOLOGIE DE CONTENEURISATION

Docker Desktop for Windows

SERVEUR WEB UTILISÉ

Nginx (image officielle Docker Hub)

🎯 **Objectif personnel :**

Ce projet a été réalisé dans le but de découvrir concrètement le fonctionnement d'un environnement Linux et des conteneurs Docker.

L'objectif était de comprendre les bases de l'administration système et réseau, de manipuler des outils professionnels, et de préparer l'entrée en formation TSSR avec une première expérience pratique.

Ce travail m'a permis de mieux comprendre le rôle d'un administrateur système et les étapes nécessaires à la mise en place d'un service web.

🛠️ **Mode de réalisation :**

Ce projet a été réalisé en **autonomie complète**.

Les différentes étapes ont été mises en place progressivement, avec recherche d'informations techniques, tests successifs et résolution d'erreurs rencontrées.

Chaque problème rencontré a été analysé, puis corrigé après plusieurs essais, ce qui a permis d'améliorer la compréhension des outils utilisés.

Cette démarche progressive m'a permis de développer une capacité d'adaptation et de diagnostic technique.

A — Vérification de WSL

Confirmer que WSL est bien installé et identifier les distributions disponibles

ÉTAPE A — Vérification de l'installation WSL

PowerShell Windows

PROGRAMME PowerShell

TERMINAL PowerShell Windows (Démarrer → PowerShell)

Avant toute manipulation, on vérifie que WSL (Windows Subsystem for Linux) est bien installé sur le système. Cette commande liste toutes les distributions Linux disponibles et indique leur version (WSL1 ou WSL2) ainsi que leur état (Running / Stopped).

```
PowerShell                                                    wsl

PS C:\Users\user> wsl --list --verbose

# Résultat attendu :
  NAME      STATE           VERSION
* Ubuntu    Running         2
```

✅ **Résultat attendu :** La distribution Ubuntu apparaît dans la liste, avec la version `2` indiquant WSL2. L'astérisque (*) indique la distribution par défaut.

```
PS C:\WINDOWS\system32> wsl --list --verbose
  NAME              STATE           VERSION
* docker-desktop    Stopped         2
  Ubuntu            Stopped         2
PS C:\WINDOWS\system32>
```

Fig. A — Résultat de `wsl --list --verbose` dans PowerShell confirme la présence d'Ubuntu WSL2

B Lancement d'Ubuntu

Ouvrir une session Linux Ubuntu depuis Windows

ÉTAPE B Démarrage de la session Ubuntu WSL

PowerShell → Terminal Ubuntu

PROGRAMME PowerShell

TERMINAL PowerShell Windows puis terminal Ubuntu

On lance une session Ubuntu directement depuis PowerShell. Une fois la commande exécutée, on se retrouve dans un terminal Linux authentique. Le prompt change et affiche un chemin Linux.

PowerShell — wsl

```
PS C:\Users\user> wsl -d Ubuntu

# Le prompt Linux s'affiche :
user@MACHINE-NAME:/mnt/c$ ← on est maintenant dans Ubuntu
```

✅ **Résultat attendu :** Le prompt change et affiche quelque chose du type `user@MACHINE:/mnt/c$`. On est désormais dans un environnement Linux sous Windows.

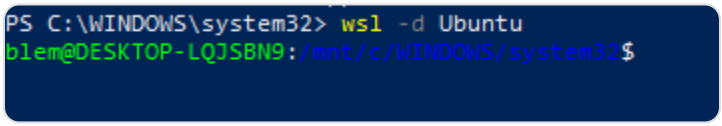


Fig. B — Le prompt Linux `user@machine:/mnt/c$` confirme l'entrée dans l'environnement Ubuntu WSL2

C

Informations de base Linux

Identifier l'utilisateur, le répertoire courant et le contenu du dossier

ÉTAPE C

Premières commandes Linux — whoami, pwd, ls

Terminal Ubuntu (WSL)

PROGRAMME **Ubuntu / terminal WSL**

TERMINAL **Terminal Ubuntu (démarré à l'étape B)**

On exécute trois commandes fondamentales pour s'orienter dans l'environnement Linux. **whoami** affiche le nom de l'utilisateur courant. **pwd** (print working directory) affiche le chemin complet du répertoire dans lequel on se trouve. **ls** liste le contenu du répertoire courant.

Bash — Ubuntu WSL linux

```
# Afficher l'utilisateur courant
$ whoami
user

# Afficher le répertoire courant
$ pwd
/home/user

# Lister le contenu du dossier courant
$ ls
Documents  Downloads  Desktop ...
```

✅ **Résultat attendu :** Le nom d'utilisateur Linux, le chemin du répertoire home, et la liste des dossiers/fichiers présents dans le home s'affichent correctement.

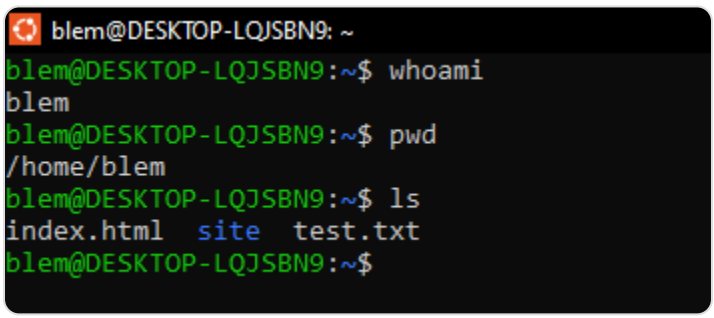


Fig. C — Résultats des commandes `whoami`, `pwd` et `ls` dans le terminal Ubuntu WSL2

D

Mise à jour du système Linux

Mettre à jour les paquets et le système Ubuntu

ÉTAPE D

apt update & apt upgrade — Mise à jour du système

Terminal Ubuntu (WSL)

PROGRAMME **Ubuntu / terminal WSL**

TERMINAL **Terminal Ubuntu**

Avant d'installer quoi que ce soit, il est indispensable de mettre à jour le système. **sudo apt update** récupère la liste des dernières versions disponibles de tous les paquets sans rien installer. **sudo apt upgrade** installe les mises à jour disponibles. Le mot-clé `sudo` exécute la commande avec les droits administrateur (superutilisateur).

⚠️ Durant `sudo apt upgrade`, une confirmation peut être demandée. Taper **Y** puis **Entrée** pour confirmer l'installation des mises à jour.

Bash — Ubuntu WSL apt

```
# Étape 1 : Mettre à jour la liste des paquets
$ sudo apt update
Get:1 http://archive.ubuntu.com/ubuntu jammy InRelease [270 kB]
...
Reading package lists... Done

# Étape 2 : Installer les mises à jour disponibles
$ sudo apt upgrade
Do you want to continue? [Y/n] Y
...
Processing triggers for libc-bin ... done.
```

✅ **Résultat attendu :** Le système Ubuntu est mis à jour sans erreur. Le message final indique que les paquets sont traités avec succès.

```
blem@DESKTOP-LQJSBN9:~$ sudo apt update
[sudo] password for blem:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1875 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [342 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.9 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1663 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [322 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [34.2 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:15 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7368 B]
Get:16 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [13.1 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:18 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Get:19 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1576 kB]
Get:20 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [252 kB]
Get:21 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:22 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.7 kB]
Get:23 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1167 kB]
Get:24 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:25 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [22.8 kB]
Get:26 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2757 kB]
Get:27 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [642 kB]
Get:28 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [208 B]
Get:29 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Fetched 11.7 MB in 3s (3477 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
blem@DESKTOP-LQJSBN9:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
blem@DESKTOP-LQJSBN9:~$
```

Fig. D — Mise à jour réussie du système Ubuntu avec `sudo apt update` *et* `sudo apt upgrade`

E

Vérification de la version du noyau Linux

Afficher les informations techniques sur le noyau utilisé

ÉTAPE E

uname -a — Informations noyau Linux

Terminal Ubuntu (WSL)

PROGRAMME **Ubuntu / terminal WSL**

TERMINAL **Terminal Ubuntu**

La commande **uname -a** affiche toutes les informations sur le noyau Linux actif. Elle permet de confirmer que le noyau WSL2 est bien utilisé (la mention `microsoft-standard-WSL2` ou similaire apparaît dans la sortie).

Bash — Ubuntu WSL

uname

```
$ uname -a
Linux MACHINE 5.15.146.1-microsoft-standard-WSL2 #1 SMP Thu ...
x86_64 x86_64 x86_64 GNU/Linux
```

✅ **Résultat attendu :** La sortie contient `microsoft-standard-WSL2` confirmant l'utilisation du noyau Linux WSL2. L'architecture `x86_64` est également visible.

```
blem@DESKTOP-LQJSBN9:~$ uname -a
Linux DESKTOP-LQJSBN9 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC
blem@DESKTOP-LQJSBN9:~$
```

Fig. E — Résultat de `uname -a` confirmant le noyau Linux WSL2

F1

Vérification de la distribution Linux

Identifier la version exacte d'Ubuntu installée

ÉTAPE F1

lsb_release -a — Informations distribution Ubuntu

Terminal Ubuntu (WSL)

PROGRAMME **Ubuntu / terminal WSL**

TERMINAL **Terminal Ubuntu**

La commande **lsb_release -a** affiche les informations détaillées sur la distribution Linux installée : nom, version, codename. Cette information est utile pour connaître la version exacte d'Ubuntu utilisée.

Bash — Ubuntu WSL

lsb_release

```
$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.3 LTS
Release:        22.04
Codename:       jammy
```

✅ **Résultat attendu :** Ubuntu 22.04 LTS (ou version similaire), avec le codename jammy. Ces informations confirment quelle version de Linux est installée dans WSL2.

```
blem@DESKTOP-LQJSBN9:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.4 LTS
Release:        24.04
Codename:       noble
blem@DESKTOP-LQJSBN9:~$
```

Fig. F1 — Résultat de `lsb_release -a` : distribution Ubuntu 22.04 LTS

F2

Démarrage de Docker Desktop

Lancer et vérifier que le moteur Docker est actif avant toute commande Docker

ÉTAPE F2

Lancement de Docker Desktop sur Windows

Interface graphique Windows

PROGRAMME **Docker Desktop**

ACTION **Lancement et vérification du statut**

Avant d'exécuter toute commande Docker dans un terminal, il est **indispensable** de démarrer Docker Desktop. Sans cela, le moteur Docker n'est pas actif et les commandes `docker` retourneront une erreur de connexion au daemon.

⚠️ **Prérequis obligatoire :** Docker Desktop doit afficher le statut **"Engine running"** (barre verte en bas à gauche) avant de passer aux étapes suivantes.

Actions à réaliser

```
# 1. Ouvrir Docker Desktop
Menu Démarrer → Docker Desktop
(ou double-cliquer sur l'icône du bureau)

# 2. Attendre le démarrage du moteur
Patientez que l'icône Docker (baleine) apparaisse dans la barre des tâches

# 3. Vérifier le statut
✓ Engine running — Docker est prêt
```

✅ **Résultat attendu :** Docker Desktop est ouvert, le moteur est actif (statut "Engine running"), et l'icône Docker est visible dans la barre des tâches Windows. Le système est prêt pour les commandes Docker.

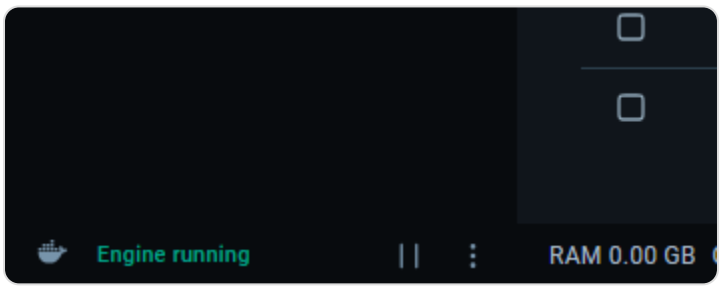


Fig. F2 — Docker Desktop lancé et moteur actif, prêt pour l'utilisation

G

Test de l'installation Docker — hello-world

Vérifier que Docker fonctionne correctement avec le conteneur de démonstration

ÉTAPE G

docker run hello-world — Test fonctionnel Docker

PowerShell Windows

PROGRAMME **PowerShell (ou Docker Desktop ouvert en arrière-plan)**

TERMINAL **PowerShell Windows**

ℹ️ Assurez-vous que **Docker Desktop** est bien lancé et en cours d'exécution avant de taper cette commande. L'icône Docker doit être visible dans la barre des tâches Windows.

La commande **docker run hello-world** est le test officiel de Docker. Elle télécharge automatiquement l'image `hello-world` depuis Docker Hub (si elle n'est pas déjà présente localement), crée un conteneur, l'exécute, et affiche un message de confirmation.

PowerShell docker

```
PS C:\> docker run hello-world

Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

✅ **Résultat attendu :** L'image est téléchargée puis le conteneur affiche `Hello from Docker!` suivi d'un message de confirmation. Docker fonctionne correctement.

```
PS C:\WINDOWS\system32> docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Fig. G — Message `Hello from Docker!` dans PowerShell : Docker est correctement installé et fonctionnel

H Vérification de la version Docker

Afficher les versions client et serveur de Docker installées

ÉTAPE H **docker version — Versions Client & Server**

PowerShell Windows

PROGRAMME **PowerShell**

TERMINAL **PowerShell Windows**

La commande **docker version** affiche deux blocs : le *Client* (outil en ligne de commande installé sur le poste) et le *Server* (moteur Docker qui tourne en arrière-plan). Cela permet de vérifier que les deux composants sont opérationnels.

PowerShell docker

```
PS C:\> docker version

Client: Docker Engine - Community
 Version:           24.0.7
 API version:       1.43
 OS/Arch:           windows/amd64

Server: Docker Desktop
 Engine:
  Version:          24.0.7
  Status:           running
```

✅ **Résultat attendu :** Les versions Client et Server s'affichent toutes les deux sans erreur, avec un statut indiquant que le moteur Docker est actif.

```
PS C:\WINDOWS\system32> docker version
Client:
 Version:           28.0.1
 API version:       1.48
 Go version:        go1.23.6
 Git commit:        068a01e
 Built:             Wed Feb 26 10:41:52 2025
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.39.0 (184744)
 Engine:
  Version:          28.0.1
  API version:      1.48 (minimum version 1.24)
  Go version:       go1.23.6
  Git commit:       bbd0a17
  Built:            Wed Feb 26 10:41:16 2025
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          1.7.25
  GitCommit:        bcc810d6b9066471b0b6fa75f557a15a1cbf31bb
 runc:
  Version:          1.2.4
  GitCommit:        v1.2.4-0-g6c52b3f
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

Fig. H — Résultat de `docker version` : versions client et serveur de Docker

I Informations détaillées Docker

Explorer la configuration complète de l'installation Docker

ÉTAPE I docker info — Configuration et environnement Docker

PowerShell Windows

PROGRAMME **PowerShell**

TERMINAL **PowerShell Windows**

La commande **docker info** affiche une grande quantité d'informations techniques sur l'installation Docker : nombre de conteneurs en cours d'exécution, nombre d'images stockées, pilote de stockage, réseau, et bien plus. Utile pour diagnostiquer un problème ou documenter l'environnement.

PowerShell docker

```
PS C:\> docker info

Client:
 Context:    default
 Debug Mode: false

Server:
 Containers: 3
  Running:   1
  Paused:    0
  Stopped:   2
 Images:     4
 Server Version: 24.0.7
 Storage Driver: overlay2
```

✅ **Résultat attendu :** Informations Docker complètes affichées sans message d'erreur. On peut lire le nombre de conteneurs, images, la version du serveur et le pilote de stockage utilisé.

```
 Version:          0.19.0
 GitCommit:        de40ad0
PS C:\WINDOWS\system32> docker info
Client:
 Version:    28.0.1
 Context:    desktop-linux
 Debug Mode: false
```

Fig. I — Résultat de `docker info` : configuration complète de l'environnement Docker

J Vérification des images Docker

Lister les images Docker disponibles localement

ÉTAPE J **docker images — Liste des images locales**

PowerShell Windows

PROGRAMME **PowerShell**

TERMINAL **PowerShell Windows**

La commande **docker images** liste toutes les images Docker téléchargées et stockées localement sur le poste. On y voit le nom de l'image, le tag (version), l'identifiant unique, la date de création et la taille.

```
PowerShell                                                docker

PS C:\> docker images

REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    a8758716bb6a   2 weeks ago    142MB
ubuntu        latest    174c8c134b2a   4 weeks ago    77.9MB
hello-world   latest    d2c94e258dcb   8 months ago   13.3kB
```

✅ **Résultat attendu :** La liste affiche au minimum l'image `hello-world` téléchargée lors de l'étape G. Les images `ubuntu` et `nginx` apparaissent également selon l'avancement du projet.

```
PS C:\WINDOWS\system32> docker images
REPOSITORY    TAG       IMAGE ID       CREATED       SIZE
nginx         latest    0cf1d6af5ca7   12 days ago   161MB
hello-world   latest    e2ac70e7319a   13 days ago   10.1kB
ubuntu        latest    f794f40ddfff   5 weeks ago   78.1MB
PS C:\WINDOWS\system32>
```

Fig. J — Résultat de `docker images` : images hello-world, ubuntu et nginx visibles

K1

Conteneurs Docker actifs — docker ps

Afficher uniquement les conteneurs en cours d'exécution

ÉTAPE K1

docker ps — Liste des conteneurs actifs

PowerShell Windows

PROGRAMME **PowerShell**

TERMINAL **PowerShell Windows**

La commande **docker ps** (sans option) affiche uniquement les conteneurs **actuellement en cours d'exécution**. Elle permet de vérifier rapidement quels conteneurs sont actifs, sur quels ports ils écoutent, et depuis combien de temps ils tournent.

PowerShell docker

```
PS C:\> docker ps

CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS   NAMES
# (aucun conteneur actif — hello-world s'est arrêté après exécution)
```

✅ **Résultat attendu :** La liste affiche les conteneurs actifs. Seuls les conteneurs ayant le statut `Up` apparaissent. Si aucun conteneur n'est en cours d'exécution, la liste est vide.

```
PS C:\WINDOWS\system32> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Fig. K1 — Résultat de `docker ps` : conteneurs en cours d'exécution

K2

Tous les conteneurs Docker — docker ps -a

Afficher tous les conteneurs, y compris ceux qui sont arrêtés

ÉTAPE K2

docker ps -a — Liste complète des conteneurs

PowerShell Windows

PROGRAMME **PowerShell**

TERMINAL **PowerShell Windows**

L'option **-a** (all) de `docker ps` affiche *tous* les conteneurs, y compris ceux qui sont arrêtés (`Exited`). C'est la commande de référence pour avoir une vue complète de l'historique des conteneurs créés.

PowerShell docker

```
PS C:\> docker ps -a

CONTAINER ID   IMAGE         COMMAND    CREATED          STATUS
c3d4e5f6a7b8   hello-world   "/hello"   10 minutes ago   Exited (0) 10 minutes ago
```

✅ **Résultat attendu :** La liste affiche le conteneur `hello-world` créé à l'étape G, avec le statut `Exited (0)` indiquant qu'il s'est terminé correctement. D'autres conteneurs apparaîtront au fur et à mesure des étapes suivantes.

```
PS C:\WINDOWS\system32> docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED        STATUS                     PORTS                NAMES
11656ad37407   hello-world   "/hello"                 2 minutes ago  Exited (0) 2 minutes ago                        wizardly_nobel
53bf35b29376   nginx         "/docker-entrypoint.…"   4 days ago     Exited (0) 4 days ago                           strange_kowalevski
5366cab28769   nginx         "/docker-entrypoint.…"   4 days ago     Exited (0) 4 days ago                           jovial_brahmagupta
```

Fig. K2 — Résultat de `docker ps -a` : liste complète avec statuts

K3

Arrêt d'un conteneur Docker — docker stop

Arrêter proprement un conteneur en cours d'exécution

ÉTAPE K3

docker stop — Arrêt propre d'un conteneur

PowerShell Windows

PROGRAMME **PowerShell**

TERMINAL **PowerShell Windows**

La commande **docker stop** envoie un signal d'arrêt (`SIGTERM`) au conteneur, lui laissant le temps de s'arrêter proprement avant de forcer l'arrêt après un délai de 10 secondes. L'identifiant du conteneur peut être récupéré via `docker ps`. Cette commande sera utilisée concrètement lors du déploiement Nginx (étape P) pour gérer les conteneurs actifs.

ℹ️ À ce stade, aucun conteneur n'est en cours d'exécution (hello-world s'arrête automatiquement). L'exemple ci-dessous illustre la syntaxe de la commande, qui sera utilisée avec les conteneurs Nginx dans les étapes suivantes.

PowerShell docker

```
# Syntaxe de la commande docker stop
PS C:\> docker stop <CONTAINER_ID>

# Exemple concret (sera utilisé avec Nginx plus tard) :
PS C:\> docker stop a1b2c3d4e5f6
a1b2c3d4e5f6

# Vérification : le conteneur n'apparaît plus dans docker ps
PS C:\> docker ps
CONTAINER ID   IMAGE   STATUS
```

✅ **Résultat attendu :** Docker retourne l'identifiant du conteneur arrêté. Un `docker ps` confirme que le conteneur n'apparaît plus dans la liste des conteneurs actifs.

L Lancement d'un conteneur Ubuntu interactif

Entrer dans un conteneur Ubuntu et ouvrir un shell bash

ÉTAPE L docker run -it ubuntu bash — Conteneur interactif

PowerShell Windows

PROGRAMME PowerShell

TERMINAL PowerShell Windows (puis shell dans le conteneur)

La commande **docker run -it ubuntu bash** lance un conteneur Ubuntu en mode interactif. `-i` garde le flux d'entrée ouvert, `-t` alloue un terminal (TTY), et `bash` est la commande exécutée à l'intérieur du conteneur. Une fois lancé, on obtient un prompt root à l'intérieur du conteneur.

PowerShell docker

```
PS C:\> docker run -it ubuntu bash

# On entre dans le conteneur, le prompt change :
root@a1b2c3d4e5f6:/# ← on est maintenant DANS le conteneur Docker Ubuntu
```

✅ **Résultat attendu :** Le prompt affiche `root@CONTAINER_ID:/#` indiquant qu'on est maintenant connecté en tant que root à l'intérieur du conteneur Ubuntu. C'est un environnement complètement isolé.

```
PS C:\WINDOWS\system32> docker run -it ubuntu bash
root@59cf5f8c0435:/#
```

Étape L — Résultat exécution

M

Mise à jour et installation dans le conteneur

Installer l'éditeur nano à l'intérieur du conteneur Ubuntu

ÉTAPE M

apt update & apt install nano — Dans le conteneur

Shell bash du conteneur Ubuntu Docker

PROGRAMME **Conteneur Ubuntu Docker (démarré à l'étape L)**

TERMINAL **Shell bash à l'intérieur du conteneur (root@id:/#)**

ℹ️ On est maintenant DANS le conteneur Docker. Les commandes suivantes s'exécutent dans l'environnement isolé du conteneur, pas sur le PC Windows. On n'utilise pas `sudo` car on est déjà root.

Les images Docker officielles sont minimales : elles ne contiennent que le strict minimum. On commence par mettre à jour les index des paquets (`apt update`), puis on installe l'éditeur de texte `nano` qui sera utilisé à l'étape suivante pour créer un fichier.

Bash — DANS le conteneur Ubuntu Docker apt

```
# On est dans le conteneur : prompt root@id:/#
root@a1b2c3d4:/# apt update
Get:1 http://archive.ubuntu.com/ubuntu focal InRelease
Reading package lists... Done

root@a1b2c3d4:/# apt install nano
Do you want to continue? [Y/n] Y
Setting up nano ... done.
```

✅ **Résultat attendu :** Nano est installé avec succès dans le conteneur. Le message `Setting up nano ... done.` confirme l'installation.

```
root@59cf5f8c0435: /
root@59cf5f8c0435:/# apt update
Get:1 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [3499 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages [1808 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1504 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [19.3 MB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [34.8 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1985 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [331 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble/restricted amd64 Packages [117 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2369 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [38.1 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2152 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [3669 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [49.5 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [36.1 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Packages [695 B]
Fetched 37.5 MB in 6s (6267 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
10 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@59cf5f8c0435:/# apt install nano
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  hunspell
The following NEW packages will be installed:
  nano
0 upgraded, 1 newly installed, 0 to remove and 10 not upgraded.
Need to get 282 kB of archives.
After this operation, 856 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 nano amd64 7.2-2ubuntu0.1 [282 kB]
Fetched 282 kB in 1s (447 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package nano.
(Reading database ... 4381 files and directories currently installed.)
Preparing to unpack .../nano_7.2-2ubuntu0.1_amd64.deb ...
Unpacking nano (7.2-2ubuntu0.1) ...
Setting up nano (7.2-2ubuntu0.1) ...
update-alternatives: using /bin/nano to provide /usr/bin/editor (editor) in auto mode
update-alternatives: warning: skip creation of /usr/share/man/man1/editor.1.gz because associated file /usr/share/man/man1/nano.1.gz (of link group edi
update-alternatives: using /bin/nano to provide /usr/bin/pico (pico) in auto mode
update-alternatives: warning: skip creation of /usr/share/man/man1/pico.1.gz because associated file /usr/share/man/man1/nano.1.gz (of link group pico
root@59cf5f8c0435:/#
```

Étape M — Résultat exécution

N

Création d'un fichier texte dans le conteneur

Utiliser nano pour créer et enregistrer un fichier dans le conteneur

ÉTAPE N

nano test.txt — Création d'un fichier dans le conteneur

Shell bash du conteneur Ubuntu Docker

PROGRAMME **Conteneur Ubuntu Docker (shell ouvert à l'étape L)**

TERMINAL **Shell bash dans le conteneur (root@id:/#)**

On crée un fichier texte à l'intérieur du conteneur en utilisant l'éditeur **nano**. Cela démontre qu'on peut travailler avec des fichiers dans un environnement conteneurisé. Nano s'ouvre dans le terminal, on saisit le texte, puis on enregistre avec **Ctrl+O** (confirmer avec Entrée) et on quitte avec **Ctrl+X**.

💡 Dans nano : **Ctrl+O** pour sauvegarder (Write Out), **Entrée** pour confirmer le nom de fichier, puis **Ctrl+X** pour quitter l'éditeur.

Bash — DANS le conteneur Ubuntu Docker

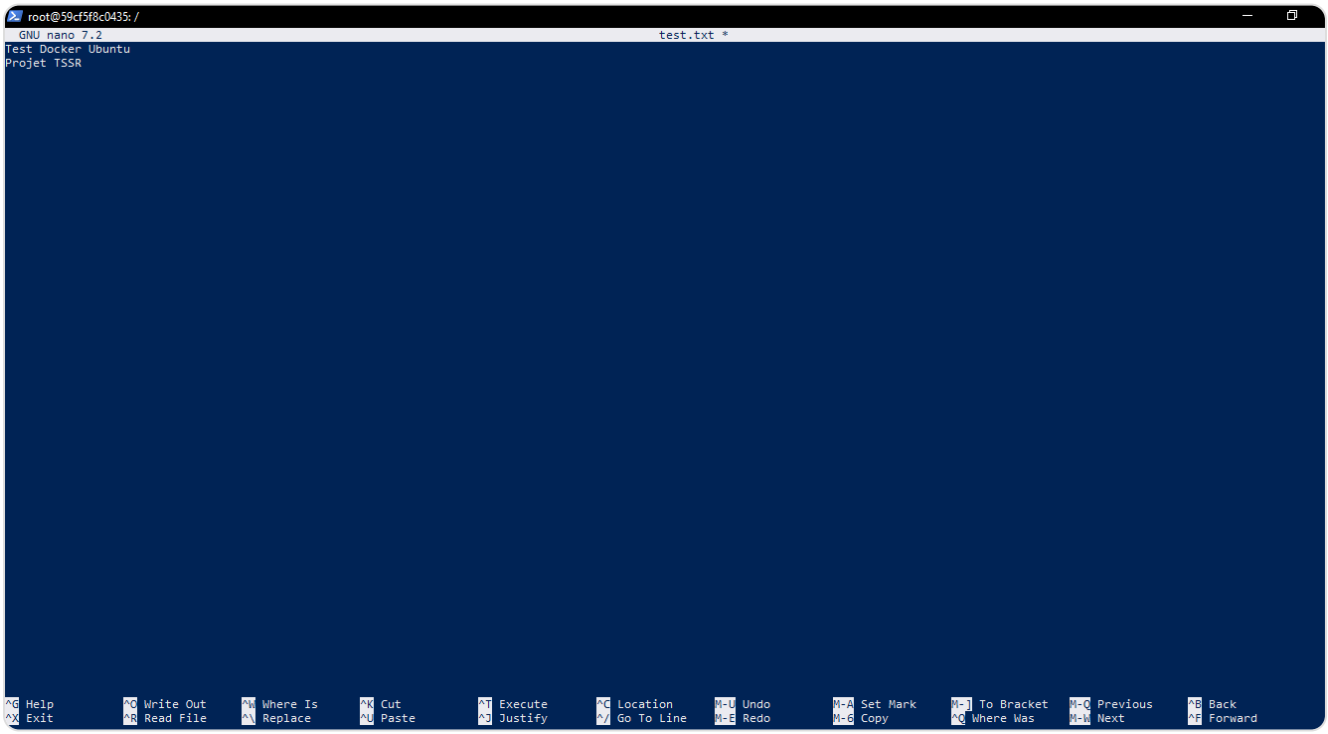
nano

```
root@a1b2c3d4:/# nano test.txt

# L'éditeur nano s'ouvre. Taper le texte suivant :
Test Docker Ubuntu
Projet TSSR

# Sauvegarder : Ctrl+O → Entrée → Ctrl+X pour quitter
[ Wrote 2 lines ]
```

✅ **Résultat attendu :** Nano confirme l'enregistrement avec le message `Wrote 2 lines`. Le fichier `test.txt` existe maintenant dans le répertoire `/` du conteneur.



Étape N — Résultat exécution

0 Lecture du fichier dans le conteneur

Afficher le contenu du fichier texte créé dans le conteneur

ÉTAPE 0 cat test.txt — Lecture du fichier dans le conteneur

`Shell bash du conteneur Ubuntu Docker`

PROGRAMME Conteneur Ubuntu Docker

TERMINAL Shell bash dans le conteneur (root@id:/#)

La commande **cat** (concatenate) affiche le contenu d'un fichier texte dans le terminal. On vérifie ici que le fichier `test.txt` créé à l'étape précédente a bien été sauvegardé correctement dans le conteneur.

Bash — DANS le conteneur Ubuntu Docker `cat`

```
root@a1b2c3d4:/# cat test.txt

Test Docker Ubuntu
Projet TSSR

# Sortir du conteneur
root@a1b2c3d4:/# exit
exit
# Retour au prompt PowerShell Windows
```

✅ **Résultat attendu :** Le contenu du fichier s'affiche tel qu'il a été saisi : `Test Docker Ubuntu` sur la première ligne et `Projet TSSR` sur la deuxième.

```
root@59cf5f8c0435:/# cat test.txt
Test Docker Ubuntu
Projet TSSR
root@59cf5f8c0435:/#
```

Étape O — Résultat exécution

P Déploiement d'un serveur web Nginx

Lancer un conteneur Nginx et accéder à la page de bienvenue depuis le navigateur

ÉTAPE P docker run -d -p 8081:80 nginx — Lancement du serveur web

PowerShell Windows

PROGRAMME **PowerShell**

TERMINAL **PowerShell Windows (quitter le conteneur de l'étape L avec `exit`)**

On lance un serveur web Nginx dans un conteneur Docker. `-d` (detached) fait tourner le conteneur en arrière-plan. `-p 8081:80` expose le port 80 du conteneur sur le port 8081 du PC local (translation de port). `nginx` est le nom de l'image utilisée.

PowerShell docker

```
# Si on est encore dans un conteneur, quitter d'abord :
root@id:/# exit

# De retour dans PowerShell, lancer Nginx :
PS C:\> docker run -d -p 8081:80 nginx
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2
```

✅ **Résultat attendu :** Un identifiant de conteneur (hash long) s'affiche, indiquant que Nginx tourne en arrière-plan. Aucun message d'erreur ne doit apparaître.

```
PS C:\WINDOWS\system32> docker stop 7c9d5ddcc58c
7c9d5ddcc58c
PS C:\WINDOWS\system32> docker run -d -p 8081:80 nginx
9a8d1002a98895f1f6d2db8a9329f299c629feadafb0d903b44de4168aacb3e9
PS C:\WINDOWS\system32>
```

Fig. P.1 — Conteneur Nginx lancé : identifiant du conteneur affiché dans PowerShell

Q — Création du dossier du site personnalisé

Préparer le dossier local qui contiendra la page HTML personnalisée

ÉTAPE Q — mkdir & notepad — Création du dossier et du fichier HTML

PowerShell Windows

PROGRAMME PowerShell

TERMINAL PowerShell Windows

On crée un dossier `C:\site-tssr` sur le disque Windows qui servira de volume Docker. Ce dossier sera monté dans le conteneur Nginx à l'étape S. La commande `mkdir` crée le dossier et `notepad` ouvre le fichier `index.html` dans le Bloc-notes Windows pour l'éditer.

PowerShell — mkdir · notepad

```
# Créer le dossier du site
PS C:\> mkdir C:\site-tssr
    Directory: C:\
Mode    LastWriteTime  Length Name
d----   ...            site-tssr

# Ouvrir le fichier index.html dans le Bloc-notes
PS C:\> notepad C:\site-tssr\index.html
```

✅ **Résultat attendu :** Le dossier `C:\site-tssr` est créé. Le Bloc-notes s'ouvre avec un fichier vide nommé `index.html` prêt à être édité.

```
PS C:\WINDOWS\system32> mkdir C:\site-tssr


    Répertoire : C:\


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        06/04/2026     12:07                site-tssr


PS C:\WINDOWS\system32>
```

Fig. Q — Création du dossier `C:\site-tssr` et ouverture du fichier `index.html` dans le Bloc-notes

R Création de la page HTML personnalisée

Écrire et sauvegarder le fichier index.html du site TSSR

ÉTAPE R Édition de C:\site-tssr\index.html dans le Bloc-notes

Bloc-notes Windows (Notepad)

PROGRAMME Bloc-notes Windows (Notepad)

FICHIER À ÉDITER C:\site-tssr\index.html

Dans le Bloc-notes ouvert à l'étape précédente, on saisit le code HTML de la page web personnalisée. Ce fichier sera servi par Nginx à l'étape S. On doit copier *exactement* ce code, puis sauvegarder avec **Fichier → Enregistrer** ou **Ctrl+S**.

⚠️ Lors de l'enregistrement dans le Bloc-notes, vérifier que le fichier est sauvegardé avec l'encodage **UTF-8** et que le nom se termine bien par `.html` (pas `.html.txt`).

C:\site-tssr\index.html

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8" />
  <title>Projet TSSR</title>
</head>
<body>
  <h1>Projet Docker TSSR</h1>
  <p>Serveur web fonctionnel avec Docker et Nginx.</p>
</body>
</html>
```

✅ **Résultat attendu :** Le fichier `index.html` est sauvegardé dans `C:\site-tssr\`. On peut vérifier sa présence avec `dir C:\site-tssr` dans PowerShell.

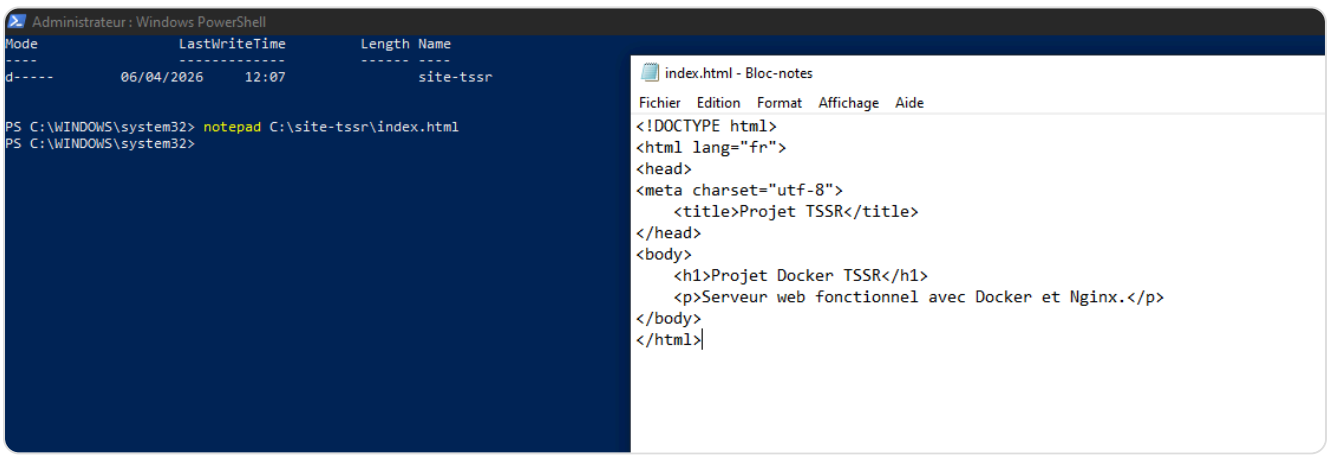


Fig. R — Fichier `index.html` édité dans le Bloc-notes Windows et sauvegardé dans `C:\site-tssr\`

S

Déploiement de la page personnalisée avec Nginx

Monter le dossier local dans Nginx et afficher la page HTML personnalisée

ÉTAPE S

docker run avec volume -v — Nginx + site personnalisé

PowerShell Windows

PROGRAMME **PowerShell**

TERMINAL **PowerShell Windows**

Cette commande est la plus complète du projet. Elle lance un conteneur Nginx qui va servir notre page HTML personnalisée. L'option **-v C:\site-tssr:/usr/share/nginx/html** monte le dossier Windows `C:\site-tssr` dans le chemin `/usr/share/nginx/html` à l'intérieur du conteneur : c'est l'emplacement par défaut où Nginx cherche les fichiers à servir. Le port `8082` est utilisé pour ne pas entrer en conflit avec le conteneur Nginx de l'étape P (port 8081).

PowerShell — docker · volume

```
PS C:\> docker run -d
  -p 8082:80
  -v C:\site-tssr:/usr/share/nginx/html
  nginx

# En une seule ligne :
PS C:\> docker run -d -p 8082:80 -v C:\site-tssr:/usr/share/nginx/html nginx
b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2
```

✅ **Résultat attendu :** Un identifiant de conteneur s'affiche. Le conteneur Nginx démarre et monte le dossier `C:\site-tssr` pour servir notre page HTML personnalisée sur le port 8082.

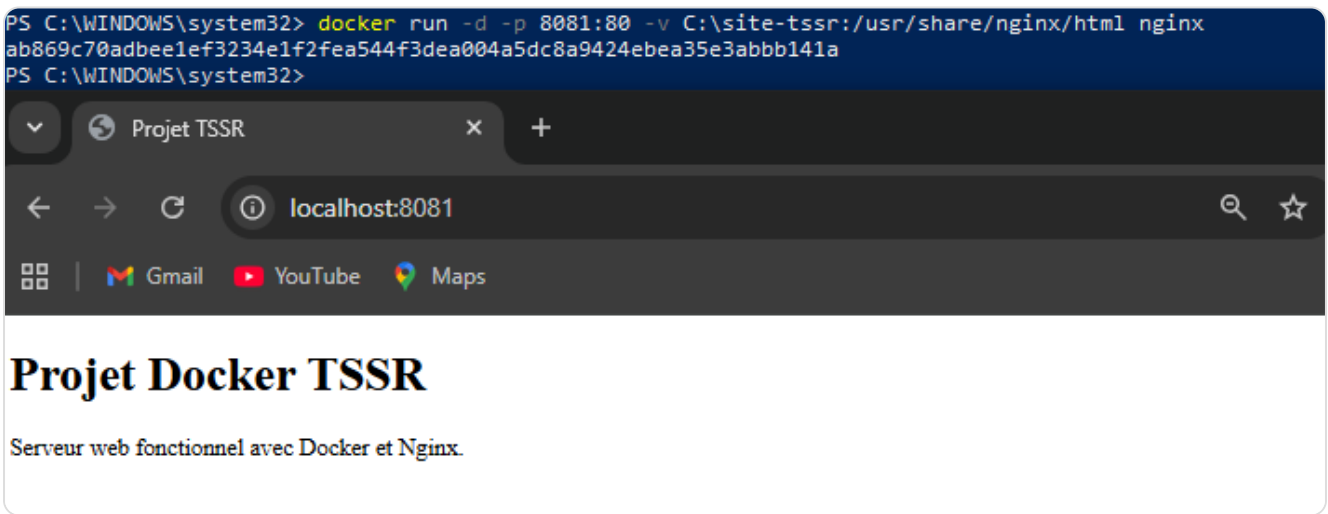


Fig. S.1 — Lancement du conteneur Nginx avec montage de volume dans PowerShell

SECTION ANNEXE — Environnement Personnel de Travail

Outils présents sur le poste personnel — ne pas confondre avec le projet TSSR ci-dessus.

Environnement technique personnel

Aperçu des outils installés sur le poste de travail personnel

Note importante — À lire avant de consulter cette section

Cette section présente l'environnement technique personnel utilisé comme contexte global de travail. Elle **ne signifie pas** que l'ensemble de ces outils a été mobilisé dans le projet principal décrit ci-dessus. Seuls **WSL2, Ubuntu, Docker Desktop, PowerShell et le Bloc-notes** ont été directement utilisés dans le cadre du projet TSSR documenté.

WSL2
Sous-système Linux pour Windows
✅ UTILISÉ PROJET

Ubuntu
Distribution Linux dans WSL2
✅ UTILISÉ PROJET

Docker Desktop
Conteneurisation Windows
✅ UTILISÉ PROJET

VS Code
Éditeur de code polyvalent
POSTE PERSO

Cursor
Éditeur IA (fork VS Code)
POSTE PERSO

Node.js
Environnement JavaScript
POSTE PERSO

Python
Langage de script polyvalent
POSTE PERSO

VirtualBox
Virtualisation de machines
POSTE PERSO

PowerShell
Terminal Windows avancé
✅ UTILISÉ PROJET

Difficultés rencontrées

Incidents techniques identifiés et solutions appliquées

Les difficultés rencontrées font partie intégrante de l'apprentissage. Identifier un problème, en comprendre la cause et trouver la solution est une compétence fondamentale en administration systèmes et réseaux.

🔴 Port déjà utilisé

Erreur système

Lors du lancement d'un conteneur Nginx avec `-p 8081:80`, Docker renvoie une erreur indiquant que le port 8081 est déjà utilisé par un autre processus ou un conteneur précédent encore actif.

Diagnostic

```
# Vérifier les ports utilisés sous Windows
PS> netstat -ano | findstr :8081
# Lister les conteneurs actifs
PS> docker ps
```

✅ **Solution :** Utiliser un port libre différent (ex. `-p 8082:80`) ou arrêter le conteneur conflictuel avec `docker stop ID`.

🟡 Erreur 403 Forbidden

Erreur HTTP

En accédant à `localhost:8082`, le navigateur affiche une erreur **403 Forbidden**. Nginx ne trouve pas de fichier `index.html` dans le dossier monté — soit le dossier est vide, soit le fichier est mal nommé.

Diagnostic

```
# Vérifier le contenu du dossier local
PS> dir C:\site-tssr
# index.html doit apparaître dans la liste
```

✅ **Solution :** Vérifier la présence de `index.html` dans `C:\site-tssr`, s'assurer qu'il n'est pas vide et que l'extension n'est pas `.html.txt`.

🔵 Chemin de volume incorrect

Erreur de configuration

Sous Windows, les chemins contenant des espaces ou des barres obliques mal formatées dans l'option `-v` de Docker provoquent une erreur de montage ou un dossier vide dans le conteneur.

Exemple

```
# ❌ À éviter (espaces dans le chemin)
-v "C:\mon site tssr:/usr/share/nginx/html"

# ✅ Correct (chemin sans espace)
-v C:\site-tssr:/usr/share/nginx/html
```

✅ **Solution :** Nommer le dossier sans espace : `site-tssr` plutôt que `mon site tssr`. Toujours utiliser des tirets à la place des espaces dans les noms de dossiers.

Compétences acquises

Bilan des savoir-faire développés au cours du projet TSSR

Linux
Navigation, commandes de base, gestion de fichiers, terminal
Bases acquises

Docker
Images, conteneurs, volumes, mapping de ports
Découverte approfondie

Conteneurs
Cycle de vie, isolation, création, suppression
En progression

Réseau
Ports, translation, localhost, HTTP
Première approche

Serveur Web
Nginx, déploiement, configuration de base
En progression

HTML
Structure de page, balises, fichier web fonctionnel
Maîtrise basique

Diagnostic
Identification d'erreurs, lecture de logs, résolution
Première approche

WSL2
Linux sur Windows, gestion des distributions
Bases acquises

Glossaire technique

Définitions des termes techniques utilisés dans ce document

WSL (Windows Subsystem for Linux)

Couche de compatibilité Windows permettant d'exécuter un environnement Linux natif directement sous Windows, sans machine virtuelle traditionnelle.

WSL2

Deuxième version de WSL utilisant un vrai noyau Linux virtualisé. Plus performant que WSL1, il permet une compatibilité complète avec les outils Linux.

Docker

Plateforme de conteneurisation permettant d'empaqueter une application et ses dépendances dans un conteneur isolé, portable et léger.

Conteneur

Instance isolée et exécutable créée à partir d'une image Docker. Chaque conteneur possède son propre système de fichiers, réseau et processus.

Image Docker

Modèle en lecture seule contenant le système de fichiers et la configuration nécessaires pour créer un conteneur. Téléchargeable depuis Docker Hub.

Docker Hub

Registre public en ligne hébergeant des milliers d'images Docker officielles et communautaires (nginx, ubuntu, hello-world, etc.).

Volume (-v)

Mécanisme Docker permettant de monter un dossier du système hôte à l'intérieur d'un conteneur, assurant la persistance et le partage de données.

Port Mapping (-p)

Translation de port entre l'hôte et le conteneur. Par exemple, `-p 8081:80` redirige le port 8081 du PC vers le port 80 du conteneur.

Nginx

Serveur web haute performance, souvent utilisé comme serveur HTTP, reverse proxy ou load balancer. Disponible comme image officielle Docker.

Daemon Docker

Service en arrière-plan (dockerd) qui gère les conteneurs, images, volumes et réseaux. Démarré automatiquement par Docker Desktop.

SIGTERM

Signal d'arrêt propre envoyé par `docker stop`. Laisse au processus le temps de se terminer correctement avant un arrêt forcé (SIGKILL).

sudo

Commande Linux permettant d'exécuter une instruction avec les droits administrateur (superutilisateur/root). Indispensable pour les opérations système.

apt (Advanced Package Tool)

Gestionnaire de paquets Debian/Ubuntu. `apt update` met à jour la liste des paquets, `apt install` installe un logiciel.

localhost

Adresse réseau (127.0.0.1) qui désigne la machine locale. Utilisée pour accéder aux services hébergés sur son propre ordinateur.

✓ Conclusion

Bilan global du projet et perspectives pour la formation TSSR

🎯 Bilan du projet

Ce projet a constitué une découverte concrète et pratique des technologies fondamentales de l'administration système et réseau. En partant d'un environnement Windows standard, l'ensemble des étapes a permis de maîtriser progressivement l'utilisation d'un terminal Linux réel grâce à WSL2, de comprendre le fonctionnement de la conteneurisation avec Docker, et de déployer un serveur web Nginx fonctionnel capable de servir une page HTML personnalisée.

La résolution des difficultés rencontrées — conflits de ports, erreur 403, chemin de volume incorrect — a permis de développer une première approche **méthodique et rigoureuse du diagnostic technique**, compétence centrale dans les métiers de l'administration systèmes et réseaux.

🐧 **Environnement Linux**
Terminal Linux authentique sur Windows via WSL2 : commandes, navigation, gestion de fichiers.

🐳 **Docker**
Gestion d'images, création et gestion de conteneurs, volumes et mapping de ports.

⚙️ **Serveur web Nginx**
Déploiement d'un serveur HTTP conteneurisé, accessible localement via le navigateur.

📄 **Site personnalisé**
Création et mise en ligne d'une page HTML personnalisée servie par Nginx.

🔍 **Diagnostic**
Identification et résolution de problèmes techniques réels rencontrés.

🎓 **Base TSSR**
Ce projet pose les fondations concrètes pour la suite de la formation TSSR.

Ce travail constitue une **première étape solide et opérationnelle** vers les métiers de l'administration système et réseau. Il pose les bases nécessaires — maîtrise du terminal Linux, compréhension de la conteneurisation, déploiement d'un service réseau — pour aborder la formation TSSR dans les meilleures conditions.

📖 Glossaire technique

Définitions des termes techniques utilisés dans ce document

WSL `Windows Subsystem for Linux`

Couche de compatibilité développée par Microsoft permettant d'exécuter un environnement Linux directement sous Windows, sans machine virtuelle traditionnelle. WSL2 utilise un vrai noyau Linux.

Docker

Plateforme de conteneurisation qui permet d'empaqueter une application et ses dépendances dans un conteneur léger, portable et isolé du système hôte.

Conteneur `container`

Instance d'exécution isolée créée à partir d'une image Docker. Un conteneur a son propre système de fichiers, réseau et processus, mais partage le noyau du système hôte.

Image Docker

Modèle en lecture seule contenant le système de fichiers et la configuration nécessaires pour créer un conteneur. Les images sont téléchargées depuis Docker Hub.

Docker Hub

Registre public en ligne hébergeant des images Docker prêtes à l'emploi (nginx, ubuntu, hello-world...). C'est le "store" officiel des images Docker.

Volume `-v`

Mécanisme de montage permettant de partager un dossier entre le système hôte (Windows) et l'intérieur d'un conteneur Docker. Utilisé pour servir des fichiers personnalisés.

Port mapping `-p`

Translation de port qui redirige le trafic réseau d'un port du PC local vers un port à l'intérieur du conteneur. Exemple : `-p 8081:80` redirige le port 8081 vers le port 80 du conteneur.

Nginx

Serveur web open source haute performance, utilisé pour servir des pages HTML, agir comme proxy inverse ou load balancer. L'image officielle est disponible sur Docker Hub.

Daemon Docker

Processus serveur qui tourne en arrière-plan et gère les conteneurs, images, réseaux et volumes. Docker Desktop lance automatiquement le daemon au démarrage.

SIGTERM

Signal d'arrêt propre envoyé par `docker stop` au conteneur, lui laissant le temps de terminer ses opérations en cours avant l'arrêt forcé (après 10 secondes par défaut).

Bash

Shell (interpréteur de commandes) par défaut sur la plupart des distributions Linux. C'est le programme qui interprète les commandes saisies dans le terminal.

sudo `superuser do`

Commande Linux qui exécute une instruction avec les droits administrateur (root). Nécessaire pour les opérations système comme l'installation de paquets.

apt `Advanced Package Tool`

Gestionnaire de paquets utilisé par Ubuntu et Debian pour installer, mettre à jour et supprimer des logiciels. `apt update` actualise la liste, `apt install` installe.

Terminal

Interface en ligne de commande permettant d'interagir avec le système d'exploitation en saisissant des commandes textuelles. PowerShell (Windows) et Bash (Linux) en sont des exemples.